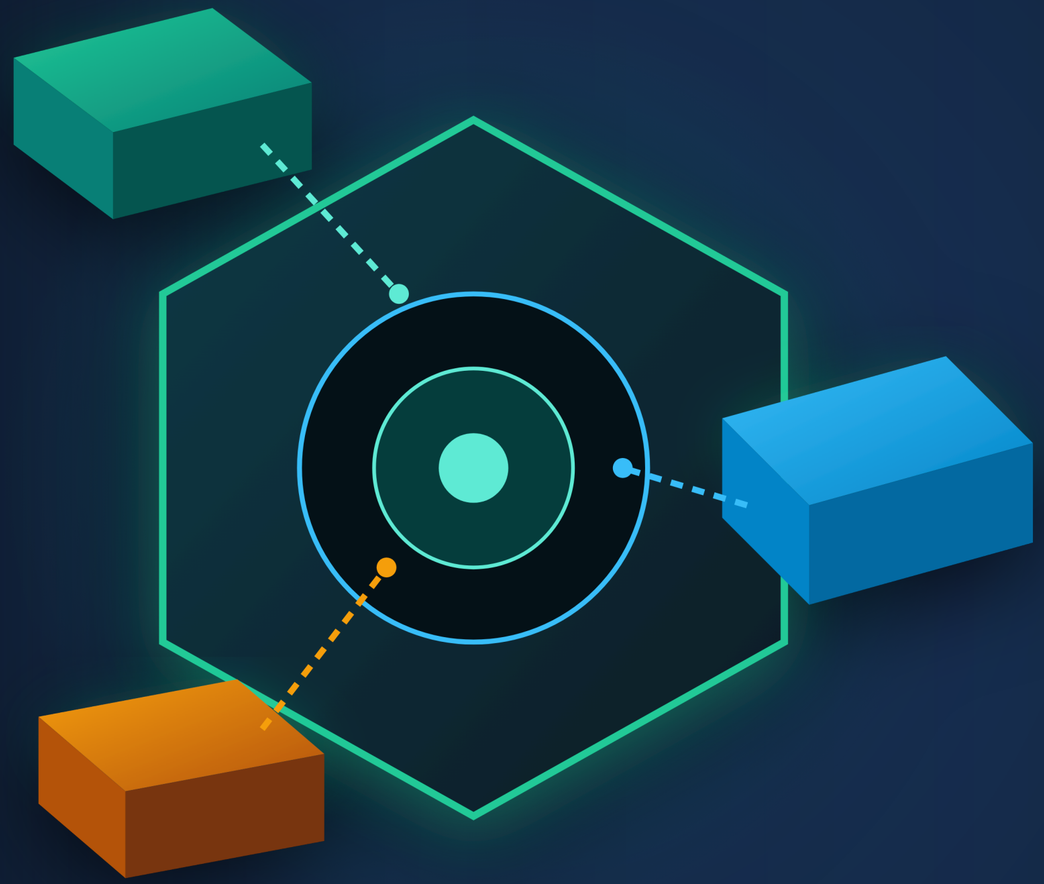
Software Engineering Lab Lab 16

Dependency Injection di Go

Constructor Injection vs Service Locator:
Trade-off Konkret untuk Testability



Lukman / lukman-ss

Coupling Mengunci Gateway di Domain

Processor yang membuat RealGateway sendiri tidak bisa diuji tanpa jaringan.

MASALAH STRUKTURAL

Processor meng-instantiate &RealGateway{} secara internal.
Swap provider gateway memaksa edit domain code.
Test menjadi integration test lambat dan flaky.

Dalam kode tradisional, domain langsung membangun dependensi konkret. Lab ini menunjukkan bahwa pola tersebut mengunci pengujian pada infrastruktur nyata — bukan berarti setiap sistem tanpa DI gagal production, melainkan trade-off antara kemudahan写 dan kemampuan uji.

When a component builds its own dependencies, replacing them requires editing the component itself.

Konstruktor Tanpa Interface Menyulitkan Swap

Menambahkan parameter interface hanya mengubah signature, bukan domain logic.

BEFORE (TIGHT COUPLING)

NewProcessor() — tidak ada parameter dependency.
Setiap test harus menggunakan RealGateway asli.
Ganti provider = ubah seluruh call site.

AFTER (CONSTRUCTOR INJECTION)

NewProcessor(g PaymentGateway) — menerima interface.
Test cukup berikan MockGateway.
Ganti provider = ubah satu baris di composition root.

Perubahan dari konstruktor tanpa parameter ke konstruktor dengan interface PaymentGateway tidak mengubah satu baris pun logika bisnis di dalam ProcessPayment. Ia hanya memindahkan titik pemilihan implementasi dari dalam domain ke composition root. Ini bukan jaminan test lebih cepat, tetapi ia membuat mock substitution mungkin tanpa refaktor lebih jauh.

Injecting an interface moves the selection point from domain to the composition root.

IoC Bukan Fitur, Tapi Arah Kendali

Domain menerima dependensi, bukan memilih atau membuatnya sendiri.

PRINSIP INTI

Constructor injection memindahkan instantiation ke caller.
Processor bergantung pada PaymentGateway interface, bukan implementasi.
Komposisi terjadi sekali di composition root (main.go).

Inversion of Control adalah prinsip umum; DI adalah bentuk spesifiknya untuk dependensi. Dalam lab ini, arahnya sederhana: main.go memegang kendali atas concrete type, Processor hanya menerima interface yang sudah jadi. Domain validation dan gateway error tetap berjalan di processor, tetapi pemilihan gateway bukan urusan domain lagi.

IoC is a principle of control direction; DI is one specific mechanism to realize it for dependencies.

Alur Request dari Composition Root ke Gateway

Empat langkah: konfigurasi → injeksi → validasi → charge.

STEP 01

main.go membuat RealGateway & menyuntikkannya ke NewProcessor.

→

STEP 02

Call site memanggil ProcessPayment(100).

→

STEP 03

Processor validasi amount > 0, lalu buat Money{100, USD}.

→

STEP 04

Gateway.Charge(m) dipanggil; error dikembalikan ke caller.

Flow ini menggambarkan eksekusi konkrit dari composition root hingga gateway dipanggil. Validasi invalid amount terjadi sebelum gateway terlibat, sehingga MockGateway.ChargedMoney tetap 0 — terbukti oleh TestProcessor_InvalidAmount. Simulasi ini tidak sama dengan flow production yang memiliki retry, timeout, dan circuit breaker.

A clean request flow separates configuration, validation, and external call concerns.

Constructor Membuat Dependency Terlihat

Signature NewProcessor(g PaymentGateway) memindahkan keputusan ke caller.

```
func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}

func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Parameter constructor g PaymentGateway membuat dependency terlihat di signature publik. Domain tidak lagi perlu import RealGateway. Ini membuat test bisa memberi mock, tetapi constructor masih perlu guard jika nil dependency ingin ditolak eksplisit — saat ini tidak ada nil check.

Constructor parameters are the public contract for dependencies; missing nil guard means runtime risk remains.

Service Locator Menyembunyikan Dependency

BadProcessor menerima Container, bukan PaymentGateway langsung.

ANTI-PATTERN

NewBadProcessor(c Container) menerima abstraksi yang lebih tinggi.
ProcessPayment memanggil container.GetPaymentGateway() secara internal.
Dependensi nyata PaymentGateway tersembunyi dari signature constructor.
Test harus mock Container, bukan gateway — tambahan indirection.

BadProcessor tidak salah secara fungsional: ia tetap memanggil Charge dan menghasilkan output yang sama. Masalahnya terletak pada keterbacaan dan testability — setiap developer baru harus menelusuri container untuk memahami dependensi sebenarnya. Ini bukan berarti pattern ini selalu rusak di semua konteks, tetapi ia menambah biaya kognitif yang tidak perlu.

Hiding dependencies behind a locator increases cognitive load and makes mocking harder.

Biaya Tersembunyi: Indirection vs Eksplisitas

Konstruktor injection menambah boilerplate kecil; Service Locator menambah ketidakjelasan.

Aspek	Constructor	Service Locator
Testability	Mock langsung	Mock container + gateway
Readability	Signature jelas	Butuh telusuri container
Swap provider	Satu baris di root	Satu baris di root
Risk nil	Panic jika tanpa guard	Panic jika Get returns nil

Tabel ini membandingkan dua pola berdasarkan empat aspek yang relevan dengan perawatan kode. Constructor injection memiliki boilerplate lebih tinggi di composition root, tetapi membuat dependensi terlihat di setiap constructor call site. Service locator mengurangi jumlah parameter, tetapi menambatkan domain pada API container. Lab ini tidak mengukur overhead runtime kedua pendekatan.

The hidden cost of a pattern is often measured in maintenance time, not execution time.

Setup Pengujian: Mock Isolasi Gateway

MockGateway merekam panggilan dan bisa diforced fail tanpa jaringan.

TEST HARNESS

MockGateway menyimpan ChargedMoney dan toggle ShouldFail.
6 test cases: 3 untuk Processor (success, error, invalid), 3 untuk BadProcessor.
go test -race ./... dijalankan pada semua test sequential.
Tidak ada test concurrent atau benchmark throughput.

Test suite dirancang untuk memverifikasi tiga hal: happy path, error propagation, dan validasi pre-check. MockGateway menggantikan RealGateway sepenuhnya — tidak ada panggilan HTTP, tidak ada dependency eksternal. Race detector berjalan pada test sequential; ia bukan bukti bahwa produksi tanpa concurrent access aman, hanya menunjukkan tidak ada data race pada skenario yang diuji.

A focused test harness isolates one behavior at a time; it proves the path, not the whole system.

Semua 6 Test Pass, Race Detector Clean

Output lokal: PASS 6/6, 0.00s per test case.

```
$ go test -race -v ./tests/
=== RUN   TestProcessor_Success
--- PASS: TestProcessor_Success (0.00s)
=== RUN   TestProcessor_GatewayError
--- PASS: TestProcessor_GatewayError (0.00s)
=== RUN   TestProcessor_InvalidAmount
--- PASS: TestProcessor_InvalidAmount (0.00s)
=== RUN   TestBadProcessor_Success
--- PASS: TestBadProcessor_Success (0.00s)
=== RUN   TestBadProcessor_GatewayError
--- PASS: TestBadProcessor_GatewayError (0.00s)
=== RUN   TestBadProcessor_InvalidAmount
--- PASS: TestBadProcessor_InvalidAmount (0.00s)
PASS: ok lab16/tests (cached)
```

Enam test case berlari dalam 0.00s per kasus. Race detector tidak melaporkan warning. Ini membuktikan jalur yang diuji aman dari data race pada goroutine tunggal — bukan berarti seluruh lifecycle production bebas race.

Zero race warnings on a small sequential suite prove the tested paths; they do not prove the whole runtime safe.

Angka 0.00s Tidak Otomatis Jadi SLO Production

Waktu eksekusi test ≠ latency gateway nyata; mereka diukur di kondisi berbeda.

Metric	Nilai	Makna	Batas
Test duration	0.00s/case	Unit test in-memory	Bukan network latency
Race warnings	0	Sequential test clean	Belum concurrent test
Mock coverage	Charge captured	Path: valid+error+invalid	Belum edge: timeout
Demo output	RealGateway charging	Integration manual verified	Bukan automated claim

Tabel ini memetakan setiap angka ke interpretasi yang benar dan batasannya. Test duration 0.00s berarti operasi in-memory cepat, bukan gateway berlatensi nol. Race warnings nol berarti jalur sequential aman, bukan berarti konkurensi production aman. Cakupannya mengikuti test.

Benchmark numbers are context-bound; they measure what the harness measures, nothing more.

Trade-off: Boilerplate Konstruksi vs Kemampuan Uji

Menambah satu parameter constructor membuka jalan untuk mock isolation.

BIAYA & MANFAAT

Biaya: Tambah 1 parameter di constructor
Manfaat: Mocking tanpa network call di unit test
Biaya: Composition root perlu tahu concrete type
Manfaat: Error propagation tetap terverifikasi rapi

Trade-off ini sebanding: satu baris boilerplate di constructor mendapatkannya isolated testing. Namun mock isolation bukan berarti production safety — ia hanya menghilangkan dependency eksternal dari test. Validasi amount dan error wrapping masih membutuhkan test terpisah.

DI trades a small construction cost for isolation gains; it does not buy production safety.

Nil Guard Menutupi Runtime Risk

Menambahkan pengecekan nil pada constructor membuat dependency wajib terlihat.

```
func NewProcessor(g PaymentGateway) *Processor {
    if g == nil {
        return nil // atau panic, tergantung kebijakan tim
    }
    return &Processor{gateway: g}
}
```

Menambahkan nil check pada constructor merupakan langkah selanjutnya yang disarankan. Saat ini, constructor menerima nil tanpa peringatan — caller akan menghadapi nil pointer panic saat gateway.Charge dipanggil. Lab ini belum menerapkan guard tersebut; ia sengaja mempertahankan pola minimal untuk fokus pada kontras DI vs Service Locator.

Explicit nil checks convert runtime surprises into compile-time or call-site failures.

Sebelum vs Sesudah: Konstruktor Injection

Perubahan hanya di signature; domain logic tetap identik.

BEFORE (TIGHT COUPLING)

Processor { gateway RealGateway }
NewProcessor() → &Processor{gateway: &RealGateway{}}
Test: harus pakai RealGateway atau setup mocking kompleks
Swap: edit seluruh call site

AFTER (CONSTRUCTOR INJECTION)

Processor { gateway PaymentGateway }
NewProcessor(g PaymentGateway) → &Processor{gateway: g}
Test: cukup passed MockGateway{}
Swap: satu baris di main.go

Perbandingan struktural ini menunjukkan bahwa perubahan terbesar bukan di dalam Processor, melainkan di titik komposisinya. Domain logic tetap sama; yang berubah adalah siapa yang bertanggung jawab atas concrete type. Ini bukan berarti semua sistem harus melalui transformasi serupa — ia hanya menunjukkan trade-off yang diambil lab ini.

The biggest change is at the composition boundary, not inside the domain logic.

Apa yang Tidak Dicakup oleh Lab Ini

Laboratorium ini hanya membuktikan pola struktural, bukan skalabilitas production.

YANG DILAB

Kontras struktural antara Constructor Injection dan Service Locator.
Verifikasi mock substitution bekerja pada test sequential.
Race detector bersih pada test suite saat ini.

YANG TIDAK DILAB

Lifecycle management (singleton, scoped, transient).
Performance overhead reflection-based DI frameworks.
Concurrent access patterns pada shared dependencies.
Production defect rate reduction claim.

Lab ini sengaja membatasi diri pada pola struktural dasar. Ia tidak mengklaim bahwa constructor injection menghasilkan production-ready code tanpa pertimbangan lebih lanjut. Lifecycle management, concurrency safety, dan framework integration adalah topik yang masih perlu penyelidikan terpisah.

A lab proves what it designs to prove; it does not absolve you from production considerations.

Edge Case: Nil Dependency Menghasilkan Panic

NewProcessor(nil) lulus konstrutor; panic terjadi saat Charge dipanggil.

SKENARIO EDGE

Caller mengoper nil sebagai PaymentGateway.
Constructor tidak menolak; Processor tetap terbuat.
ProcessPayment(100) → gateway.Charge(m) → nil pointer panic.
Error tidak ter-catch; crash terjadi di runtime.

Nil dependency adalah edge case yang belum ditangani oleh constructor saat ini. Ia bukan kelemahan DI secara prinsip, melainkan konsekuensi dari ketiadaan explicit guard. Dalam production, caller yang bertanggung jawab memastikan non-nil; namun test suite saat ini tidak memiliki case untuk nil input.

Absence of a guard is not a DI flaw; it is a defensive programming gap that any pattern must address.

Test Suite Membuktikan Tiga Jalur Saja

Happy path, gateway error, dan invalid amount — tidak lebih.

YANG DIBUKTIKAN

MockGateway berhasil menangkap panggilan Charge dengan amount dan currency benar.
Error dari gateway dipropagasikan ke caller tanpa di-suppress.
Pre-check amount <= 0 mencegah panggilan gateway (ChargedMoney tetap 0).

Ketiga jalur ini membentuk inti verifikasi behavior Processor. Namun test ini membuktikan jalur A, bukan klaim production B. Misalnya, concurrency safety, memory leak, atau integration dengan payment provider sungguhan tidak tercover oleh test suite yang ada.

Tests prove the paths they cover; they do not prove the paths they do not traverse.

Yang Belum Dibuktikan oleh Test Suite

Test sequential tidak menjamin behavior pada konkurensi atau skala besar.

YANG TIDAK DIBUKTIKAN

Concurrency safety: tidak ada concurrent test yang memicu race condition.
Lifecycle management: tidak adauji singleton vs transient vs scoped.
Framework DI overhead: tidak diukur latency injection vs manual new.
Production defect reduction: tidak ada studi empiris dalam lab ini.

Ruang kosong ini bukan kegagalan lab — ia adalah batasan yang disengaja. Lab bertujuan menunjukkan kontras struktural, bukan menyelidiki aspek non-fungsional. Klaim production safety memerlukan evidence tambahan dari load test, chaos engineering, atau audit kode manual.

The absence of evidence is not evidence of absence; it is a scope declaration.

Risiko Menerapkan Pola Tanpa Pertimbangan Lifecycle

Constructor injection tanpa lifecycle management berpotensi memory leak.

RISIKO POTENSIAL

Setiap call site membuat Processor baru → object graph tumbuh tanpa batas.
Shared state di dalam PaymentGateway (misal token cache) perlu sinkronisasi.
Nil guard tidak ada → caller error menghasilkan panic di runtime.
Komposisi root tunggal (`main.go`) bisa menjadi bottleneck pada sistem besar.

Risiko-rikiko ini muncul ketika pola diterapkan secara naif tanpa mempertimbangkan scope dan lifecycle. Lab ini tidak menyelesaikan masalah-masalah tersebut; ia hanya menunjukkan bahwa pola konstruktif injection lebih mudah diuji daripada tight coupling. Solusi lengkap memerlukan dependency container atau manual wiring yang lebih matang.

Pattern application without lifecycle awareness turns a testing benefit into an operational risk.

Checklist Implementasi Dependency Injection yang Aman

Langkah-langkah praktis untuk menerapkan DI tanpa overengineering.

- ✓ [ ] Dependeansi dideklarasikan sebagai interface, bukan concrete type.
- ✓ [ ] Constructor injection digunakan; semua dependency eksplisit di signature.
- ✓ [ ] Composition root (`main.go`) adalah satu-satunya tempat konstruksi concrete.
- ✓ [ ] Value objects (seperti `Money`) diinstansiasi langsung, bukan melalui DI.
- ✓ [ ] Validasi input dilakukan sebelum delegasi ke dependency eksternal.
- ✓ [ ] `go test -race ./...` dijalankan dan PASS sebelum merge.

Checklists guide practice; they do not replace architectural judgment or runtime verification.

Anti-Pattern: Service Locator di Domain Logic

Menyuntikkan container ke kelas bisnis menyembunyikan dependensi aktual.

HINDARI POLA INI

NewBadProcessor(c Container) — signature menyembunyikan PaymentGateway.
container.GetPaymentGateway() dipanggil di dalam method domain.
Test harus mock Container alih-alih gateway — tambahan kompleksitas.
Swap provider tetap satu baris, tetapi readability dan traceability menurun.

Service Locator bukanlah pola yang salah secara absolut — ia useful dalam konteks tertentu seperti plugin systems. Namun untuk domain logic yang memerlukan testability dan traceability, pola ini menambah biaya kognitif tanpa memberikan manfaat yang setara. Lab ini menunjukkannya sebagai kontras, bukan sebagai solusi yang dilarang mutlak.

Service Locator is a tool, not a sin; misuse depends on where and why it is applied.

Reproduksi Mandiri di Mesin Lokal

Go standard library, zero external dependencies.

```
cd labs/16-dependency-injection
go test -race -v ./tests/     # validasi 6 test cases
go run ./cmd/demo            # jalankan demo kedua pola
go vet ./...                 # statik analisis tambahan
```

Lab ini zero-dependency dan dapat direproduksi langsung di mesin lokal manapun. Menjalankan test dan demo membuktikan pola struktural bekerja sesuai deskripsi. Ia tidak membuktikan bahwa pola tersebut cocok untuk sistem dengan ratusan call site atau multi-threaded production workload.

Reproducibility on a local machine is necessary but not sufficient for production confidence.

Constructor Membuat Dependency Terlihat

Signature yang eksplisit memudahkan mock, swap, dan code review.

PRINSIP REKAYASA

DI bukan fitur framework — ia arah kendali atas instantiation.
Constructor injection memindahkan keputusan ke composition root.
Service Locator mengurangi boilerplate parameter, tetapi menambah indirection.
Nil guard dan lifecycle management adalah tanggung jawab implementasi, bukan pola.

Pelajaran utama lab ini: dependency injection mengubah bagaimana kita berpikir tentang ownership. Domain tidak owning dependensinya; caller yang owning dan menyuntikkannya. Ini tidak otomatis membuat codebase production-ready, tetapi ia membuat trade-off antara testability dan complexity lebih terlihat.

DI shifts ownership from the component to its assembler; visibility improves testability.

Constructor memperlihatkan dependency wajib; validasi nil tetap tanggung jawab constructor.

Constructor injection makes dependencies explicit; it does not replace lifecycle management or nil safety.

Software Engineering Lab 16 – Dependency Injection di Go

internal/di/processor.go • internal/di/locator.go • internal/di/gateway.go
cmd/demo/main.go • tests/processor_test.go
README.md • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Sumber: commit d7fb619
Test passing: 6 test cases, 0 failures.
Race detector: clean (go test -race ./...)
Simulasi in-memory mock; bukan proof production DI governance completeness.